

Guide du pipeline MIMoSA

Que fait chaque fichier, comment le lancer, et à quoi servent les options. Écrit pour être lu simplement, sans connaissance du code.

À quoi sert le pipeline

Le pipeline transforme des images de microscopie **CZI** (le format brut, très lourd, sorti du microscope Zeiss) en une base de données **BIDS** propre : des images réduites, positionnées correctement dans l'espace, prêtes à être analysées ou empilées en volume 3D.

Le parcours d'une image

Chaque fichier CZI passe par ces étapes. Chaque flèche est un script que tu lances.

CZI (brut)	→	NIfTI réduits	<code>mimosa_hpc_converter2.py</code>
CZI (brut)	→	OME-TIFF mosaïque	<code>mimosa_czi2ometiff_converter.py</code>
NIfTI réduits	→	tranches préparées	<code>mimosa_slice_preprocessor.py</code>
tranches préparées	→	volume 3D	<code>mimosa_stacking_2D_2_3D-2.py</code>

Note : la mosaïque OME-TIFF est l'image de publication (une lame entière). Les NIfTI réduits servent à l'analyse et à la reconstruction 3D.

Partie 1 — Les scripts que tu lances

mimosa_hpc_converter2.py

Ce qu'il fait : convertit chaque CZI en images **réduites** (NIfTI ou TIFF), rangées dans derivatives/2D/downsampled/. C'est le script principal. Il lit la vraie image native (jamais la vignette de ZEN) et fabrique lui-même les versions réduites.

Comment le lancer :

```
python mimosa_hpc_converter2.py -f nii -df 4,6,8 \  
-o /chemin/BIDS -y metadata.yml \  
--block-value mean --threads 16
```

Option	Ce que ça fait	Défaut
-f	Format de sortie : tif, nii ou both. Obligatoire.	—
-df	Exposant(s) de réduction, facteur = 2^{exposant} . Un seul (4) ou une liste (4,6,8), produits en une seule lecture. Ici -df 8 = facteur 256. Obligatoire.	—
-o	Dossier racine BIDS de sortie. Obligatoire.	—
-y	Fichier metadata.yml décrivant les CZI.	metadata.yml
-original_thickness	Épaisseur d'une coupe histologique, en μm .	100
-reorient	Réorientation utilisée pour la position (sform).	none
--block-value	Comment calculer la valeur d'un bloc : decimate (garde 1 pixel) ou mean (moyenne du bloc, mieux pour la quantification).	decimate
--threads	Threads pour produire une image plus vite. Mettre au nombre de cœurs.	nb de cœurs

mimosa_czi2ometiff_converter.py

Ce qu'il fait : recolle les scènes d'un CZI en une seule grande **mosaïque OME-TIFF** (une lame entière), rangée dans sub-X/ses-Y/micr/. C'est l'image brute BIDS, plus légère que le CZI, destinée à la publication.

Comment le lancer :

```
python mimosa_czi2ometiff_converter.py -y metadata.yml \  
-bids_root /chemin/BIDS -df 8 -channels 0,1 --threads 16
```

Option	Ce que ça fait	Défaut
-y	Fichier metadata.yml. Obligatoire.	—
-bids_root	Dossier racine BIDS de sortie. Obligatoire.	—
-df	Facteur de réduction, parmi 1, 2, 4, 6, 8. Ici -df 8 = facteur 8 (différent du script NIfTI !).	8
-channels	Canaux à convertir, ex. 0 ou 0,1.	0,1
-patch_size	Taille des carrés lus dans le CZI, en pixels.	6144
--threads	Carrés lus en parallèle. Sortie identique, juste plus vite.	nb de cœurs

mimosa_slice_preprocessor.py

Ce qu'il fait : prépare les tranches 2D réduites pour l'empilage : il les **rembourse** (padding) à une taille commune et recalcule leur position, dans derivatives/2D/preproc/.

Comment le lancer :

```
python mimosa_slice_preprocessor.py -bids_root /chemin/BIDS -res 4x
```

Option	Ce que ça fait	Défaut
-bids_root	Dossier racine BIDS. Obligatoire.	—
-res	Résolution à préparer, ex. 4x. Obligatoire.	—
-padding_delta	Marge de rembourrage, en pixels.	100
-original_thickness	Épaisseur d'une coupe, en μm .	200
-reorient	Réorientation pour la position (sform).	none

mimosa_stacking_2D_2_3D-2.py

Ce qu'il fait : empile les tranches 2D préparées en un seul **volume 3D** (comme reconstituer un cerveau à partir de coupes), dans les dérivées 3D.

Comment le lancer :

```
python mimosa_stacking_2D_2_3D-2.py -bids_root /chemin/BIDS \
-res 4x -reorient x,-z,-y -original_thickness 100
```

Option	Ce que ça fait	Défaut
-bids_root	Dossier racine BIDS. Obligatoire.	—
-res	Résolution à empiler, ex. 4x. Obligatoire.	—
-reorient	Orientation du volume 3D final.	none
-original_thickness	Espacement entre coupes, en μm .	200

check_alignment.py

Ce qu'il fait : outil de **vérification**. Prend deux images de résolutions différentes et dit si leurs grilles se superposent bien. Sert à déboguer, ne fait pas partie de la conversion.

Comment le lancer :

```
python check_alignment.py image_res-6x.nii.gz image_res-8x.nii.gz
```

Il affiche « the two grids tile exactly » si tout va bien.

Partie 2 — Les fichiers internes (pas lancés à la main)

Ces fichiers ne se lancent pas seuls : ils sont utilisés par les scripts ci-dessus. Tu n'as pas besoin d'y toucher, mais voici leur rôle.

mimosa_downsample.py

Le cœur. Il décide **comment** réduire un bloc de pixels (decimate ou mean) et **où** placer le résultat dans l'espace. Ces deux règles sont ensemble ici pour qu'elles ne se contredisent jamais. Remplace l'ancien « zoom » de la librairie Zeiss.

BIDS/czi_reader.py

Lit les informations du CZI (taille des pixels, scènes, canaux, dates).

BIDS/bids_manager.py

Construit les noms et les dossiers BIDS (sub-X, ses-Y, res-Nx, etc.).

BIDS/bids_metadata.py

Écrit les fichiers JSON à côté des images et calcule la position (sform). Délègue le calcul de position à mimosa_downsample.

Rappel important : dans mimosa_hpc_convert2.py, **-df 8 veut dire facteur 256** (2 puissance 8). Dans mimosa_czi2ometiff_convert.py, **-df 8 veut dire facteur 8**. Les deux scripts n'utilisent pas la même convention — ne pas les confondre.